[Day6] 4. RAG, 도구 통합 (실습) (1)

Category	
Date	2026년 8월 19일 오후 2:39
Published	☐
module	Day 6
order	5
강의실(classroom)	sds-ax

4-1. RAG Agent 통합 (실습)

4-2. MCP 도구 통합 (실습)

4-3. 종합 시나리오 (실습)

4-1. RAG Agent 통합 (실습)



2일차 RAG 검색기를 research_agent의 실제 도구로 연결해봅니다.

3번째 세션에서 구조가 돌아가는 것을 확인했으니 이제 더미를 진짜로 바꿀 차례입니다. 먼저 research_agent의 `retrieve_docs`입니다. 2일차에 만든 벡터 DB를 어떻게 Agent에 붙일까요? 검색기를 함수로 감싸고 `@tool`을 붙입니다. 그게 전부입니다.

그러면 임베딩을 다시 해야 할까요? 그럴 필요 없습니다. 2일차에 사내 정책 문서를 청크로 나눠 임베딩하고 Chroma에 저장한 결과가 checkpoint의 `./chroma_db`에 그대로 들어 있습니다. 오늘은 데이터를 불러오기만 합니다.

한 가지 정할 것이 있습니다. 도구로 감싸는 범위를 어디까지로 할까요? 두 가지 중 고를 수 있습니다.

- 검색기만 도구로 만듭니다. 도구는 문서 조각을 돌려주고 답변은 Agent의 모델이 만듭니다.
- 2일차 RAG 체인 전체를 도구로 만듭니다. 도구가 완성된 답변을 돌려줍니다.

여기서는 검색기만 도구로 만듭니다.

왜일까요? 도구는 재료를 주고 판단은 Agent가 하는 쪽이 Agent 구조와 잘 맞고 출처를 인용할 때도 문서 조각이 그대로 있어야 자연스럽기 때문입니다. 체인 전체를 도구로 만들면 Agent는 완성된 답만 받아 출처를 다시 알아내기 어려운 편입니다.

따라하기

1. 벡터 DB를 디스크에서 불러와 검색기를 도구로 감싸고 research_agent에 연결합니다. 아래 코드를 `rag_agent.py`로 저장해 실행합니다.

```
# rag_agent.py - 2일차 벡터 DB를 research_agent의 실제 도구로 연결
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse, BedrockEmbeddings
from langchain_chroma import Chroma
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain.agents import create_agent

load_dotenv()

# 1) 2일차에서 저장해 둔 벡터 DB를 디스크에서 불러옵니다 (임베딩 재생성 없음)
#    임베딩 모델은 2일차에서 저장할 때 쓴 것과 반드시 같아야 합니다
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings,
    collection_name="sds_policies",
)
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니다."""
```

```
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block i
n content if isinstance(block, dict))
    return content

# 2) 검색기를 도구로 감쌉니다
@tool
def retrieve_docs(query: str) -> str:
    """삼성SDS 사내 규정 문서(연차, 재택근무, 출장 등)에서 관련
내용을 검색한다."""
    docs = retriever.invoke(query)
    if not docs:
        return "관련 문서를 찾지 못했습니다."
    return "\n\n".join(
        f"[출처: {d.metadata.get('source', '알 수 없음')}]
\n{d.page_content}"
        for d in docs
    )

# 3) 진짜 RAG 도구를 가진 research_agent
worker_llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:
0",
    region_name="us-east-1",
    temperature=0,
)
research_agent = create_agent(
    worker_llm,
    [retrieve_docs],
    system_prompt=(
        "너는 삼성SDS 사내 규정 문서 검색 전문가다.\n"
        "반드시 retrieve_docs로 검색한 내용에 근거해서만 답하
라.\n"
        "답변에 출처를 함께 표기하고, 검색 결과에 없는 내용은 모
른다고 답하라.\n"
        "담당 범위 밖 질문에는 '담당 범위가 아닙니다'라고만 답하
라."
```

```
    ),
    name="research_agent",
)

if __name__ == "__main__":
    result = research_agent.invoke(
        {"messages": [HumanMessage("출장 숙박비 상한이 얼마야?")]}
    )
    print(get_text(result["messages"][-1]))
```

2. 답변에 출처 파일명이 함께 표기되는지 확인하고 문서에 없는 내용을 물어 모른다고 답하는지도 함께 봅니다.

막힐 때

- 검색 결과가 항상 비어 있습니다.

  세 가지를 순서대로 확인합니다.
  `persist_directory` 경로가 실제 폴더와 맞는지, `collection_name`이 2일차 저장 당시와 같은지(`vectorstore._collection.count()`로 문서 개수를 찍으면 바로 확인됩니다), 임베딩 모델이 저장할 때와 같은지입니다.
  특히 마지막이 함정입니다. 다른 임베딩 모델로 검색하면 오류 없이 조용히 이상한 결과가 나옵니다.

- Agent가 검색 없이 일반 지식으로 답합니다.

  프롬프트에 "반드시 retrieve_docs로 검색한 내용에 근거해서만 답하라"가 빠졌는지 확인합니다. 사내 규정은 모델이 학습한 적이 없으므로 이 지시가 없으면 그럴듯한 환각이 나옵니다.

확인: 출장 숙박비 질문에 문서 근거와 출처가 함께 담긴 답변이 나오면 성공입니다.



검색기를 도구로 감싸 RAG를 서브 Agent로 통합하는 방법을 익혔습니다.

4-2. MCP 도구 통합 (실습)



4일차 MCP 서버를 data_agent의 도구로 연결해봅니다.

research_agent가 진짜 문서를 읽게 됐으니 이번에는 data_agent 차례입니다. 더미 도구를 4일차에 만든 진짜 MCP(Model Context Protocol, 도구를 표준 규약으로 제공하는 방식) 서버로 바꿉니다. 임직원, 자산, 프로젝트 데이터를 조회하는 그 서버입니다.

그런데 RAG는 `@tool` 한 줄로 끝났는데 MCP 서버는 별도 프로세스입니다.

Agent와 어떻게 이을까요? `langchain-mcp-adapters` 패키지가 그 사이를 이어 줍니다. `MultiServerMCPClient`가 MCP 서버에 접속해 도구 목록을 받아오고 LangChain 도구 객체로 바꿔 줍니다. 바뀐 도구는 Agent의 도구 목록에 그대로 넣습니다. Agent 입장에서는 MCP 도구인지 `@tool`로 만든 도구인지 구분할 필요가 없습니다. (어댑터 내용 참조)

한 가지만 달라집니다. MCP 클라이언트는 비동기(async)로 동작합니다. 그래서 도구를 가져오는 부분과 실행 부분을 `asyncio`로 감싸야 합니다.

따라하기

1. MCP 서버에 접속해 도구 목록을 받아 data_agent를 만듭니다. 아래 코드를 `mcp_agent.py`로 저장해 실행합니다.

```
# mcp_agent.py - 4일차 MCP 서버를 data_agent에 연결
import asyncio
import sys
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage
from langchain_mcp_adapters.client import MultiServerMCP
Client
from langchain.agents import create_agent

load_dotenv()

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도
하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block i
n content if isinstance(block, dict))
    return content

async def build_data_agent():
    # 4일차에서 만든 MCP 서버를 stdio 방식으로 기동하고 접속합
```

```
니다
    client = MultiServerMCPClient(
        {
            "company": {
                "command": sys.executable,
                "args": ["mcp_server.py"],
                "transport": "stdio",
            }
        }
    )
    mcp_tools = await client.get_tools()
    print("MCP 도구 목록:", [t.name for t in mcp_tools])

    worker_llm = ChatBedrockConverse(
        model="us.anthropic.claude-sonnet-4-5-20250929-v
1:0",
        region_name="us-east-1",
        temperature=0,
    )
    data_agent = create_agent(
        worker_llm,
        mcp_tools,                    # MCP 도구가 일반 도
구처럼 들어갑니다
        system_prompt=(
            "너는 삼성SDS 사내 데이터 조회 전문가다. 임직원,
자산, 프로젝트 조회만 담당한다.\n"
            "반드시 도구로 조회한 결과에 근거해 답하고, 한 문단
으로 정리해 보고하라.\n"
            "담당 범위 밖 질문에는 '담당 범위가 아닙니다'라고만
답하라."
        ),
        name="data_agent",
    )
    return data_agent

async def main():
    data_agent = await build_data_agent()
    result = await data_agent.ainvoke(
```

```
        {"messages": [HumanMessage("클라우드운영팀 GPU 서버 몇 대야?")]}
    )
    print(get_text(result["messages"][-1]))

if __name__ == "__main__":
    asyncio.run(main())
```

2. 출력된 MCP 도구 목록에 `search_employee`, `list_team_members`, `get_asset`, `list_projects`가 모두 들어 있는지 확인합니다.

막힐 때

- `get_tools`에서 멈추거나 연결 오류가 납니다.

  MCP 서버를 단독으로 먼저 실행해 봅니다(`python mcp_server.py`). 서버 자체의 오류(경로 문제나 JSON 파일 없음)가 클라이언트 쪽에서는 연결 실패로만 보입니다. `mcp_server.py`는 같은 폴더의 JSON을 읽으므로 4일차에 만든 세 파일이 `day6_practice`에 같이 있어야 합니다. `args`의 경로가 실행 위치 기준으로 맞는지도 확인합니다.

- `RuntimeError: no running event loop` 또는 동기 호출 오류.

  MCP 도구는 비동기 도구입니다. `invoke` 대신 `ainvoke`를 쓰고 전체를 `asyncio.run(...)`으로 실행합니다.
  노트북에서는 이미 이벤트 루프가 돌고 있으므로 `asyncio.run` 대신 셀에서 `await main()`을 직접 실행합니다.

- 버전에 따른 차이.

  구버전 예제는 `async with MultiServerMCPClient(...) as client:` 형태를 씁니다. 최신 버전은 위 코드처럼 `await client.get_tools()`를 씁니다.

확인: MCP 도구 네 개가 목록에 출력되고 GPU 서버 수량을 담은 답변이 나오면 성공입니다.

> 💡 MCP 도구가 Agent 입장에서는 일반 도구와 똑같이 쓰인다는 걸 확인했습니다.

4-3. 종합 시나리오 (실습)

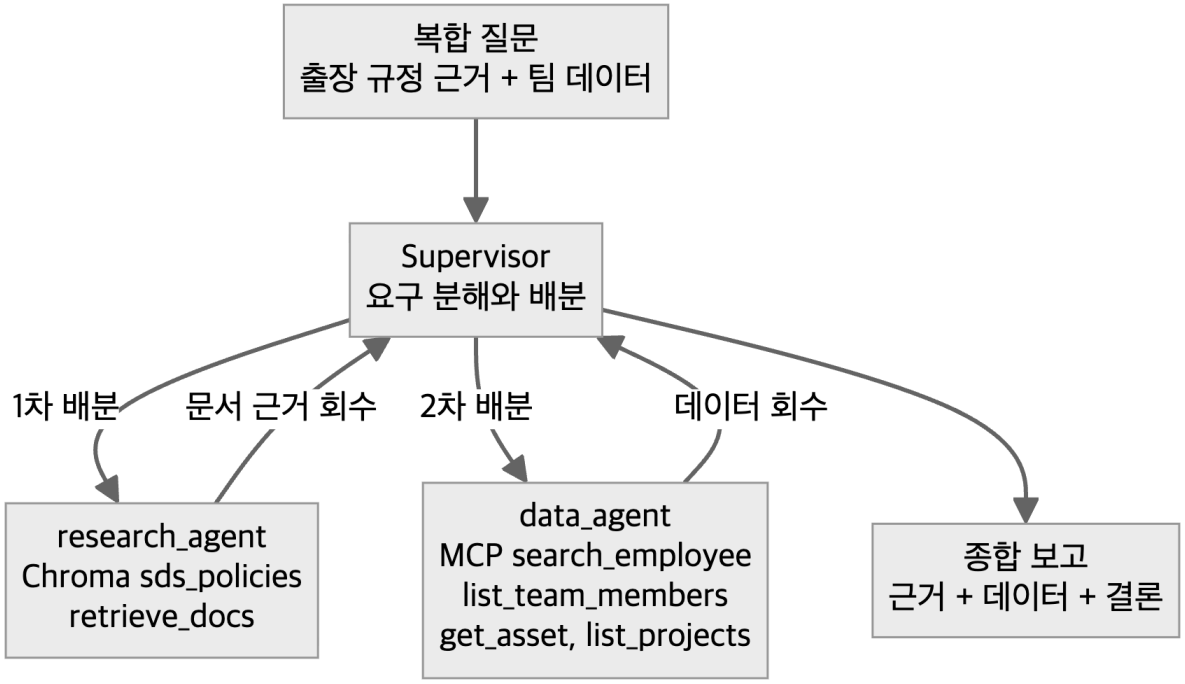
💡 문서 근거와 사내 데이터를 하나의 보고로 종합하는 최종 시나리오를 완성해봅니다.

두 Agent가 각각 진짜 도구로 답하는 것까지 확인했습니다. 이제 전부 조립합니다.

진짜 RAG를 쓰는 research_agent, 진짜 MCP를 쓰는 data_agent, 3번째 세션에서 만든 general_agent를 Supervisor로 묶습니다. 이 구조가 정말 일을 하는지 보려면 어떤 질문을 던져야 할까요? 문서도 보고 데이터도 보고 둘을 합해야 답이 나오는 질문이어야 합니다. 목표 시나리오는 다음과 같습니다.

"출장 규정에서 식비와 숙박비 근거를 찾고, 물류플랫폼팀 임직원 수와 진행 중인 프로젝트를 조회해서, 다음 분기 출장 예산 검토 보고를 종합해줘"

이 요청은 문서 근거 찾기(RAG), 사내 데이터 조회(MCP), 둘을 모아 판단하기 세 단계를 거칩니다. "중간 결과를 보고 다음을 정하는" 구조가 바로 이것입니다.



따라하기

1. 세 Agent와 Supervisor를 한 파일로 조립해 종합 시나리오를 실행합니다. 아래 코드를 `final_scenario.py`로 저장합니다.

```
# final_scenario.py - 6일차 종합: RAG + MCP + Supervisor
import asyncio
import sys
```

```
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse, BedrockEm
beddings
from langchain_chroma import Chroma
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain_mcp_adapters.client import MultiServerMCP
Client
from langchain.agents import create_agent
from langgraph_supervisor import create_supervisor

load_dotenv()
MODEL = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도
하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block i
n content if isinstance(block, dict))
    return content

def make_worker_llm():
    return ChatBedrockConverse(model=MODEL, region_name
="us-east-1", temperature=0)

# ---------- research_agent: 2일차 RAG ----------
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings,
    collection_name="sds_policies",
)
retriever = vectorstore.as_retriever(search_kwargs={"k":
```

```
3})

@tool
def retrieve_docs(query: str) -> str:
    """삼성SDS 사내 규정 문서(연차, 재택근무, 출장 등)에서 관련
내용을 검색한다."""
    docs = retriever.invoke(query)
    if not docs:
        return "관련 문서를 찾지 못했습니다."
    return "\n\n".join(
        f"[출처: {d.metadata.get('source', '알 수 없음')}]
\n{d.page_content}"
        for d in docs
    )

research_agent = create_agent(
    make_worker_llm(),
    [retrieve_docs],
    system_prompt=(
        "너는 삼성SDS 사내 규정 문서 검색 전문가다.\n"
        "반드시 retrieve_docs 결과에 근거해 답하고 출처를 표기
하라.\n"
        "담당 범위 밖 질문에는 '담당 범위가 아닙니다'라고만 답하
라."
    ),
    name="research_agent",
)

general_agent = create_agent(
    make_worker_llm(),
    [],
    system_prompt=(
        "너는 일반 응대 담당이다. 인사말과 잡담에 친절히 답하
고,\n"
        "업무 질문은 '사내 데이터 조회와 규정 검색을 도와드릴 수
있습니다'로 안내하라."
    ),
    name="general_agent",
```

```
)

async def build_app():
    # ---------- data_agent: 4일차 MCP ----------
    client = MultiServerMCPClient(
        {
            "company": {
                "command": sys.executable,
                "args": ["mcp_server.py"],
                "transport": "stdio",
            }
        }
    )
    mcp_tools = await client.get_tools()

    data_agent = create_agent(
        make_worker_llm(),
        mcp_tools,
        system_prompt=(
            "너는 삼성SDS 사내 데이터 조회 전문가다. 임직원,
자산, 프로젝트 조회만 담당한다.\n"
            "개인은 search_employee, 팀 명단과 인원수는 list
_team_members를 쓴다.\n"
            "반드시 도구로 조회한 결과에 근거해 답하고, 한 문단
으로 정리해 보고하라.\n"
            "담당 범위 밖 질문에는 '담당 범위가 아닙니다'라고만
답하라."
        ),
        name="data_agent",
    )

    # ---------- Supervisor 조립 ----------
    supervisor = create_supervisor(
        [data_agent, research_agent, general_agent],
        model=ChatBedrockConverse(model=MODEL, region_na
me="us-east-1", temperature=0),
        prompt=(
            "너는 삼성SDS 사내 어시스턴트의 Supervisor다.\n"
```

```
            "[배분 기준]\n"
            "- 임직원, 자산, 프로젝트 등 사내 시스템 데이터 조
회는 data_agent\n"
            "- 연차, 재택근무, 출장 등 사내 규정 문서 검색은 r
esearch_agent\n"
            "- 그 외 인사말, 잡담은 general_agent\n"
            "[규칙]\n"
            "- 직접 답을 지어내지 말고 반드시 담당 Agent를 통
해 확인하라.\n"
            "- 여러 요구가 섞인 질문은 각 Agent를 순서대로 호
출해 모두 처리하라.\n"
            "- 마지막에는 문서 근거와 데이터를 함께 담아 하나의
보고로 종합하고,\n"
            "  결론(가능, 불가능, 조건부)을 명시하라."
        ),
    )
    return supervisor.compile()

async def main():
    app = await build_app()
    question = (
        "출장 규정에서 식비와 숙박비 근거를 찾고, 물류플랫폼팀
임직원 수와 "
        "진행 중인 프로젝트를 조회해서, 다음 분기 출장 예산 검토
보고를 종합해줘"
    )
    print(f"질문: {question}\n")

    async for event in app.astream(
        {"messages": [HumanMessage(content=question)]},
        stream_mode="updates",
        config={"recursion_limit": 30},
    ):
        for node, update in event.items():
            for m in (update or {}).get("messages", []):
                if getattr(m, "tool_calls", None):
                    for tc in m.tool_calls:
                        print(f"[{node}] 도구: {tc['nam
```

```
e']}({tc.get('args', {})})")
                elif getattr(m, "content", None):
                    label = getattr(m, "name", None) or
node
                    print(f"[{label}] {get_text(m)[:20
0]}")

if __name__ == "__main__":
    asyncio.run(main())
```

2. 최종 출력에서 문서 출처와 사내 데이터가 모두 인용되고 마지막에 결론이 붙어 있는지 확인합니다.

막힐 때

- 어디서 실패했는지 모르겠습니다.

  조립 전 단독 테스트로 돌아갑니다. research_agent 단독(RAG Agent 통합)과 data_agent 단독(MCP 도구 통합)이 각각 성공하면 남는 문제는 Supervisor 배분뿐입니다.

- 비동기와 동기가 섞여 오류가 납니다.

  MCP가 들어오면 전체 실행을 비동기로 통일하는 편이 깔끔합니다. `invoke`는 `ainvoke`로, `stream`은 `astream`으로 바꿉니다.

- 종합 결론이 부실합니다.

  나열만 하고 판단이 없으면 Supervisor 프롬프트의 종합 지시를 구체화합니다. "마지막에 결론을 명시하라"를 추가하면 보고 품질이 올라갑니다.

확인: 두 Agent가 모두 호출되고 최종 보고에 문서 출처, 팀 데이터, 결론이 함께 담기면 성공입니다. 결론은 `[:200]` 잘림 때문에 전체 출력으로 확인합니다.



사내 데이터와 문서 근거가 하나의 보고로 종합되는 흐름을 완성했습니다.